

Claude Code en Acción. Descripción general del curso

○ **¿Qué es el Código Claude?**

Introducción

¿Qué es un asistente de codificación/programación?

Claude Code en acción

○ **Ponerse manos a la obra**

Configuración del código Claude

Configuración del proyecto

Añadiendo contexto

Realizar cambios

Encuesta de satisfacción con el curso

Contexto de control

Comandos personalizados

Servidores MCP con código Claude

Integración con GitHub

○ **Ganchos y el SDK**

Presentando los ganchos

Definiendo ganchos

Implementar un gancho

Trampas alrededor de los anzuelos

¡Ganchos muy útiles!

Otro gancho útil

El SDK de Claude Code

○ **Para finalizar**

Cuestionario sobre el Código Claude

Resumen y próximos pasos

Acerca de este curso

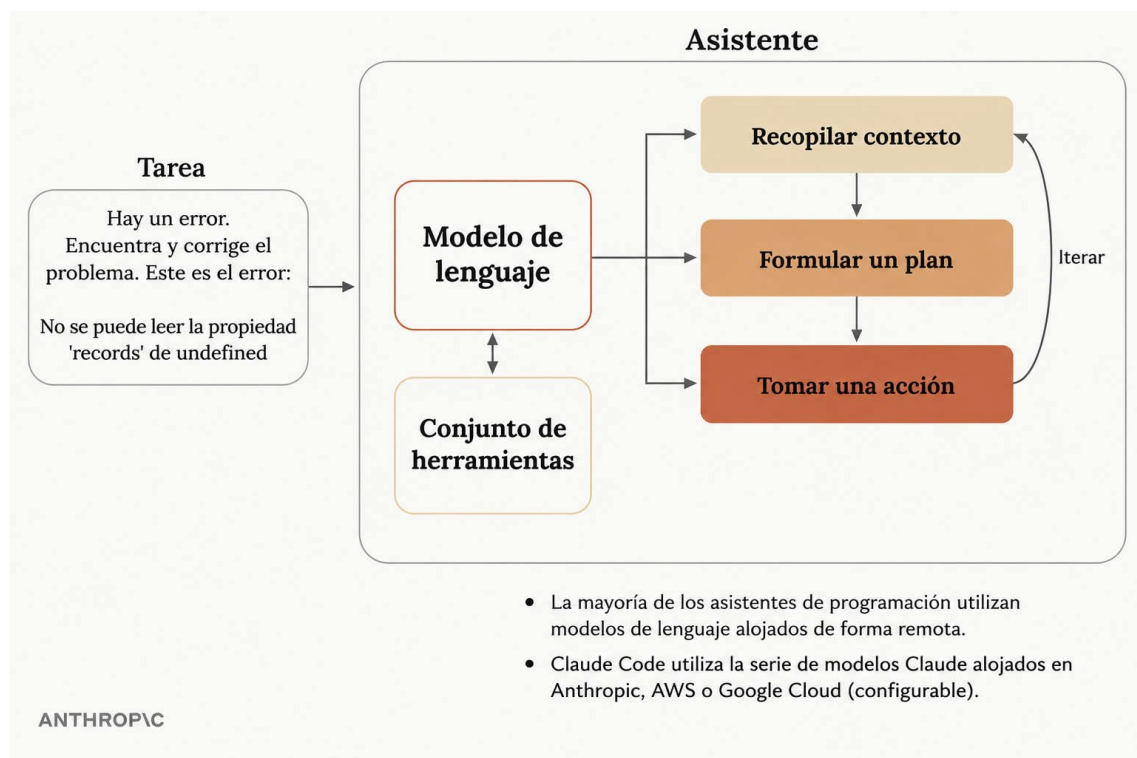
¿Qué es un asistente de codificación/programación?

Un asistente de programación es mucho más que una herramienta para escribir código: **es un sistema sofisticado que utiliza modelos de lenguaje para abordar tareas de programación complejas.**

Comprender cómo funcionan estos asistentes internamente te ayudará a apreciar lo que hace que un asistente de programación sea realmente potente.

Cómo funcionan los asistentes de codificación

Cuando se le asigna una tarea a un asistente de codificación, como corregir un error basándose en un mensaje de error, este sigue un proceso similar al que seguiría un desarrollador humano para abordar el problema:



1. **Recopilar contexto:** comprender a qué se refiere el error, qué parte del código fuente se ve afectada y qué archivos son relevantes.
2. **Formular un plan:** decidir cómo resolver el problema, como cambiar el código y ejecutar pruebas para verificar la solución.

3. **Toma medidas:** implementa la solución actualizando archivos y ejecutando comandos.

La clave reside en que los primeros y últimos pasos requieren que el asistente interactúe con el mundo exterior: leer archivos, obtener documentación, ejecutar comandos o editar código.

El desafío del uso de herramientas

Aquí es donde la cosa se pone interesante. Los modelos de lenguaje por sí solos solo pueden procesar texto y devolver texto; no pueden leer archivos ni ejecutar comandos. Si le pides a un modelo de lenguaje independiente que lea un archivo, te dirá que no tiene esa capacidad.

¿Cómo resuelven este problema los asistentes de codificación?

Utilizan un ingenioso sistema llamado "uso de herramientas".

Cómo funciona el uso de herramientas

Cuando envías una solicitud a un asistente de codificación, este agrega automáticamente instrucciones a tu mensaje que le enseñan al modelo de lenguaje cómo solicitar acciones. Por ejemplo, podría agregar un texto como: "Si quieres leer un archivo, responde con 'ReadFile: nombre del archivo'".

Aquí está el flujo completo:

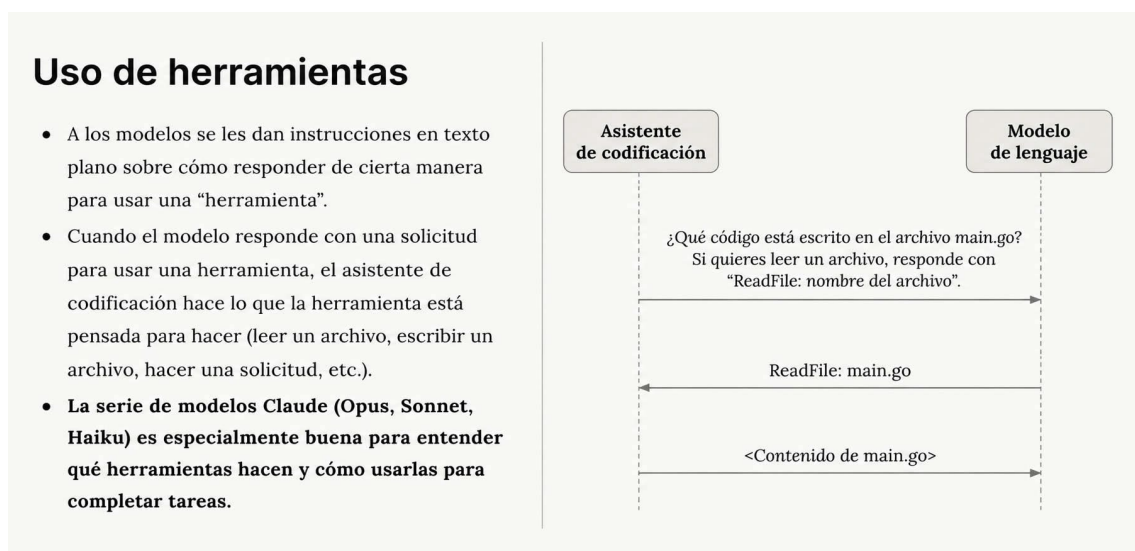
1. Preguntas: "¿Qué código está escrito en el archivo main.go?"
2. El asistente de codificación añade instrucciones de herramientas a tu solicitud.
3. El modelo de lenguaje responde: "ReadFile: main.go"
4. El asistente de codificación lee el archivo real y envía su contenido de vuelta al modelo.
5. El modelo de lenguaje proporciona una respuesta final basada en el contenido del archivo.

Este sistema permite que los modelos de lenguaje puedan "leer archivos", "escribir código" y "ejecutar comandos" de forma efectiva,

aunque en realidad solo estén generando respuestas de texto cuidadosamente formateadas.

Por qué es importante el uso de herramientas de Claude

No todos los modelos de lenguaje son igual de buenos en el uso de herramientas. La serie de modelos Claude (Opus, Sonnet y Haiku) destaca especialmente por su capacidad para comprender la función de las herramientas y utilizarlas eficazmente para completar tareas complejas.



Esta destreza en el uso de herramientas proporciona varias ventajas clave para Claude Code:

Beneficios del uso de herramientas resistentes

- **Afronta tareas más difíciles:** Claude puede combinar diferentes herramientas para manejar trabajos complejos y utilizará herramientas que no haya visto antes.
- **Plataforma extensible:** puedes añadir fácilmente nuevas herramientas a Claude Code, y Claude se adaptará para utilizarlas a medida que evolucione tu flujo de trabajo.
- **Mayor seguridad:** Claude Code puede navegar por las bases de código sin necesidad de indexación, lo que a menudo significa no enviar todo el código a servidores externos.

Conclusiones clave

Para comprender los asistentes de codificación, basta con tener en cuenta algunos puntos esenciales:

- Los asistentes de codificación utilizan modelos de lenguaje para completar diferentes tareas.
- Los modelos de lenguaje necesitan herramientas para manejar la mayoría de las tareas de programación del mundo real.
- No todos los modelos de lenguaje utilizan herramientas con el mismo nivel de habilidad.
- El uso avanzado de herramientas por parte de Claude permite una mayor seguridad, personalización y longevidad en Claude Code.

Esta capacidad de uso de la herramienta es lo que transforma un simple modelo generador de texto en un potente asistente de codificación que puede leer tus archivos, comprender tu código fuente y realizar cambios significativos en tus proyectos.

Texto vídeo:

¿Qué es un asistente de programación?

Idea principal

Un asistente de programación es un sistema basado en modelos de lenguaje que ayuda a resolver tareas de desarrollo de software de forma iterativa.

No se limita únicamente a generar texto o código:

- analiza problemas
 - recopila contexto
 - formula planes
 - ejecuta acciones
 - revisa resultados
 - vuelve a iterar si es necesario
-

Flujo de funcionamiento

El proceso mostrado en el vídeo sigue este esquema:

1. Recopilar contexto

El asistente primero intenta entender:

- el problema
- el código implicado
- los archivos relevantes
- el entorno
- el error producido

Ejemplo del vídeo

Cannot read property 'records' of undefined

Este error suele indicar que:

- se intenta acceder a records
- pero el objeto anterior es undefined

Ejemplo típico:

`response.records`

cuando realmente:

`response === undefined`

2. Formular un plan

Una vez entendido el problema, el asistente:

- identifica posibles causas
- prioriza hipótesis
- decide qué comprobar primero

Ejemplos:

- revisar la respuesta de una API
 - validar datos recibidos
 - comprobar estados nulos
 - inspeccionar logs
 - buscar errores de asincronía
-

3. Ejecutar acciones

El asistente puede:

- leer archivos
- modificar código
- ejecutar comandos
- lanzar pruebas
- usar herramientas externas

Aquí aparece un concepto importante:

Herramientas (Tools)

El modelo de lenguaje no trabaja solo.

- Se apoya en herramientas para:
- interactuar con el sistema

- acceder a archivos
 - ejecutar procesos
 - automatizar tareas
-

Iteración

El flujo no es lineal.

Después de actuar:

- el asistente evalúa resultados
- obtiene nuevo contexto
- ajusta el plan
- vuelve a intentarlo

Es un ciclo continuo:

Contexto → Plan → Acción → Nuevo contexto

Claude Code

El vídeo menciona que Claude Code utiliza modelos Claude alojados remotamente.

Puede ejecutarse sobre:

- Anthropic
- AWS
- Google Cloud

Esto permite:

- escalabilidad
 - acceso remoto
 - actualización centralizada de modelos
-

Ideas importantes del vídeo

Un asistente NO “adivina”

Trabaja mediante:

- contexto
 - razonamiento iterativo
 - uso de herramientas
-

La calidad del contexto es crítica

Cuanto mejor contexto tenga:

- mejores decisiones toma
- menos iteraciones necesita

Por eso son importantes:

- logs
 - mensajes de error
 - estructura del proyecto
 - documentación
 - archivos relevantes
-

Conceptos clave

- LLM (Large Language Model)
 - Contexto
 - Iteración
 - Tool usage
 - Planificación
 - Resolución de errores
 - Automatización de tareas
-

Resumen ultra corto

Un asistente de programación moderno:

1. entiende el problema
2. recopila contexto
3. crea un plan
4. ejecuta acciones
5. repite el proceso hasta resolverlo

La clave no es solo “generar código”, sino combinar:

- razonamiento
- contexto
- herramientas
- iteración continua.

Claude Code in Action

Claude Code incluye un conjunto completo de herramientas integradas para tareas de desarrollo comunes, como leer archivos, escribir código, ejecutar comandos y administrar directorios. Pero lo que realmente hace que Claude Code sea potente es la forma inteligente en que combina estas herramientas para abordar problemas complejos de varios pasos.

Claude Code in Action — Documentación Premium

Traducción adaptada/natural al castellano con capturas relevantes, explicaciones estructuradas y reinterpretación de las diapositivas.

00:00 — Introducción

Claude Code se presenta como un asistente de programación avanzado capaz de utilizar herramientas, ejecutar comandos y resolver tareas complejas de forma iterativa.

Tools with Claude Code

Name	Purpose
Agent	Launch a subagent to handle a task
Bash	Run a shell command
Edit	Edit a file
Glob	Find files based upon a pattern
Grep	Search the contents of a file
LS	List files and directories
MultiEdit	Make several edits at the same time
NotebookEdit	Write to a cell in a Jupyter notebook
NotebookRead	Read a cell

Continued...

Name	Purpose
Read	Read a file
TodoRead	Read one of the created to-do's
TodoWrite	Update the list of to-do's
WebFetch	Fetch from a URL
WebSearch	Search the web
Write	Write to a file

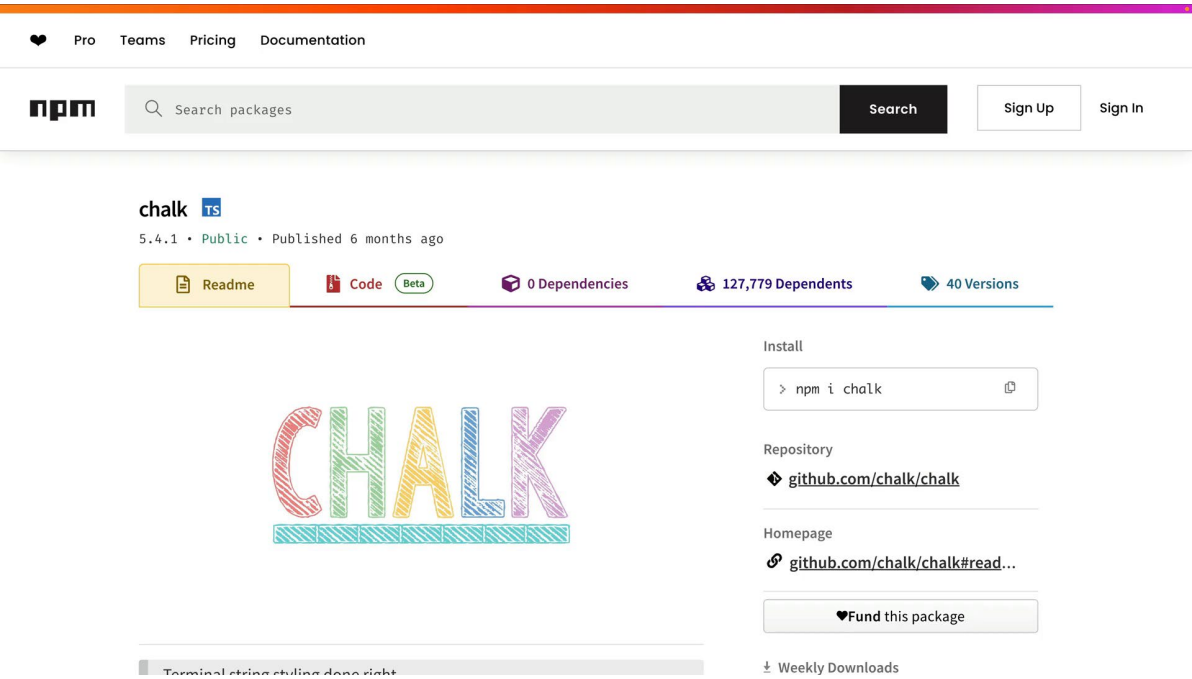
ANTHROPIC

Diapositiva traducida / resumen visual:

Claude utiliza herramientas para leer archivos, ejecutar comandos y automatizar tareas.

00:00:48 — Optimización de rendimiento

Claude analiza la librería Chalk para localizar cuellos de botella de rendimiento y aplicar optimizaciones automáticas.

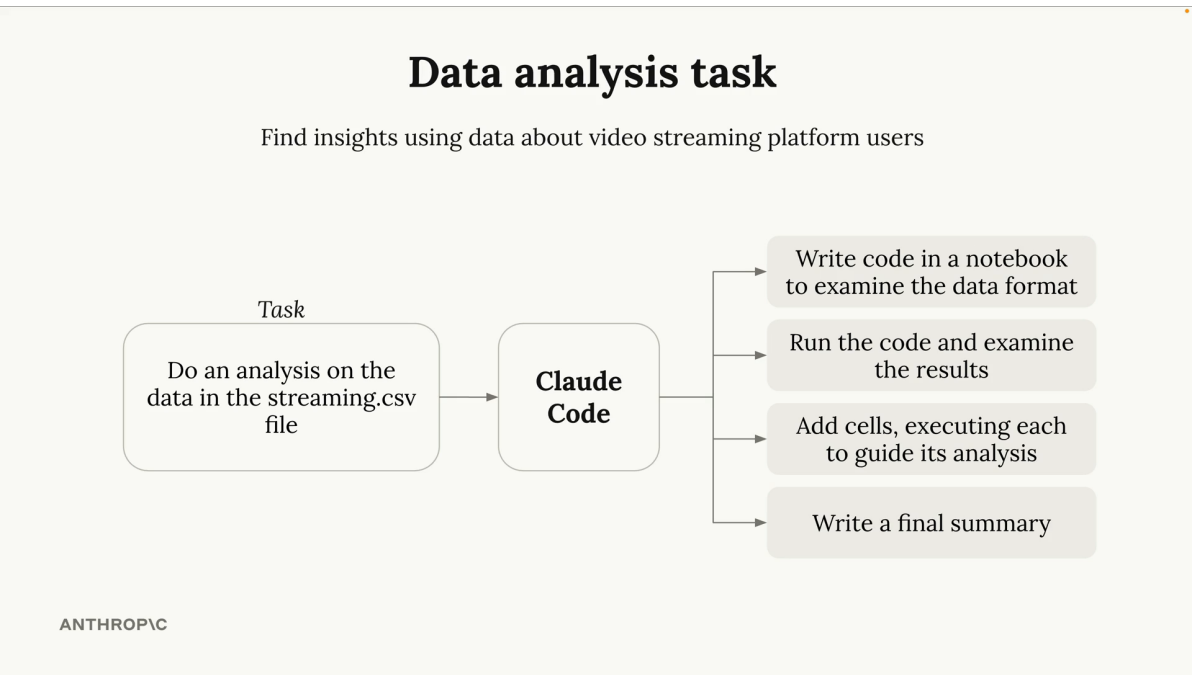


Diapositiva traducida / resumen visual:

Benchmarking y profiling automático aplicado a librerías JavaScript.

00:02:18 — Análisis de datos

Claude procesa datasets CSV para detectar patrones de churn y generar conclusiones útiles automáticamente.

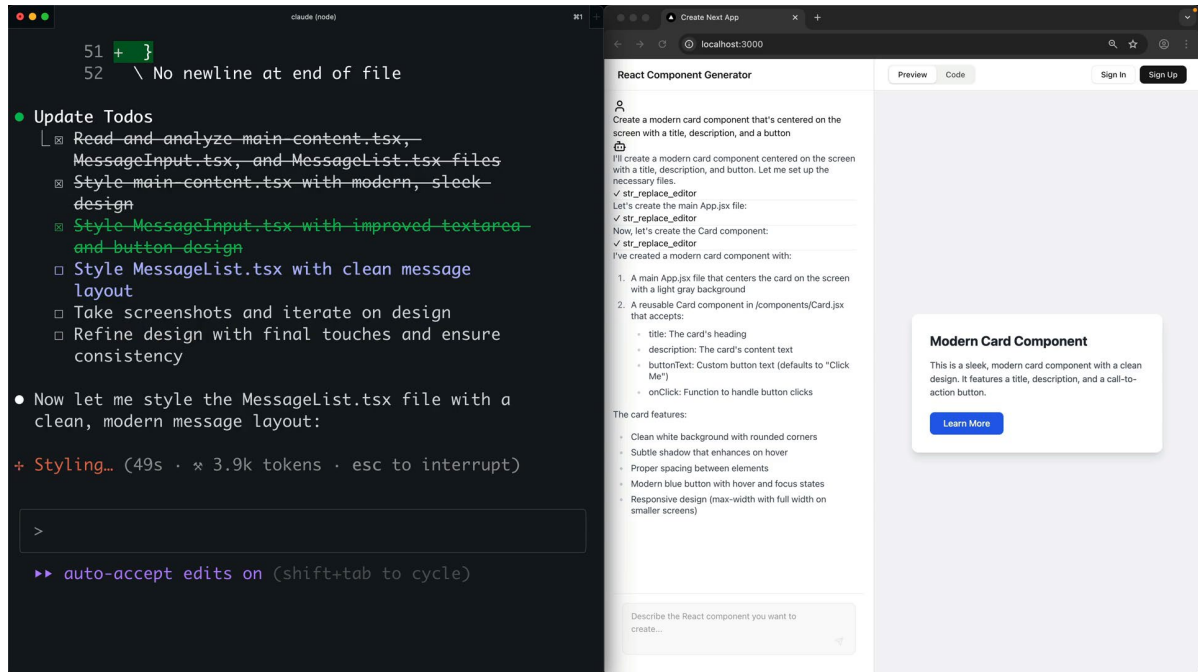


Diapositiva traducida / resumen visual:

Análisis inteligente de datasets y detección de patrones de abandono.

00:04:00 — Herramientas personalizadas

El vídeo muestra cómo extender Claude Code con nuevas herramientas e integraciones empresariales.

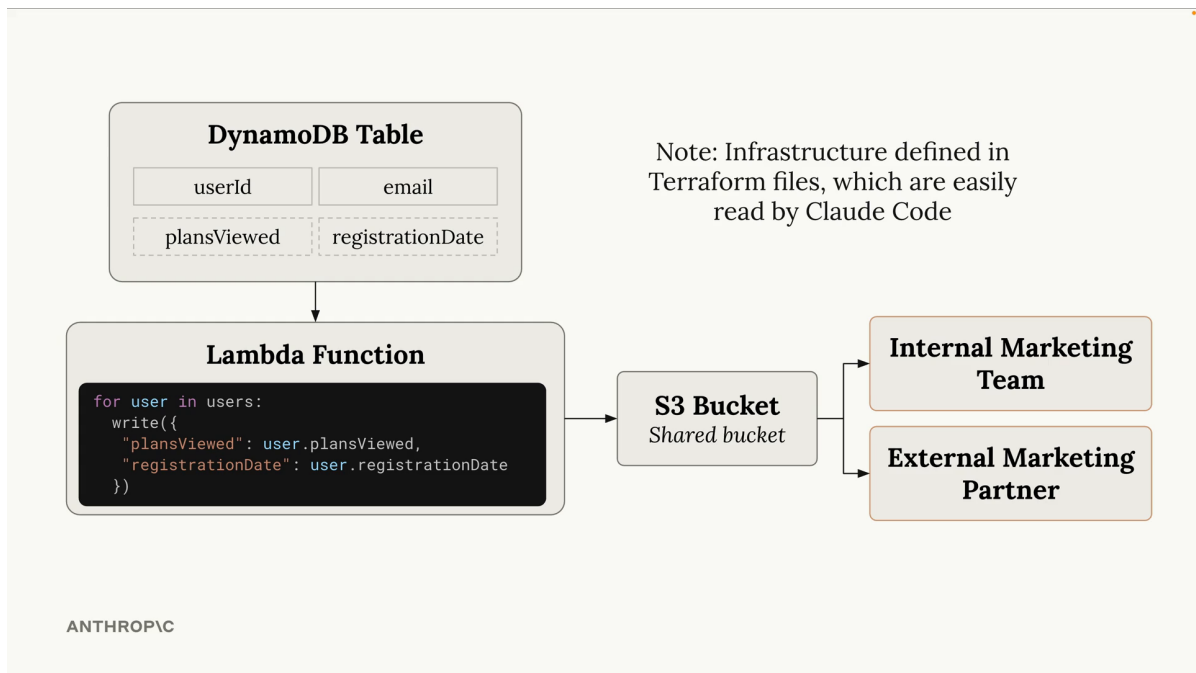


Diapositiva traducida / resumen visual:

Claude Code puede ampliarse con herramientas personalizadas.

00:05:20 — Seguridad e infraestructura

Claude revisa cambios de infraestructura y detecta una posible filtración de datos personales en AWS.



Diapositiva traducida / resumen visual:

Detección preventiva de riesgos de seguridad y exposición de datos.

00:08:11 — Conclusión

La potencia real de Claude Code surge de combinar razonamiento, herramientas y contexto.

Feat: User Processing by StephenGridler

StephenGridler wants to merge 2 commits into main from users

CRITICAL SECURITY ISSUE: PII EXPOSURE TO PARTNER ACCOUNTS

Issue: This PR introduces a serious security vulnerability by adding user email addresses (PII) to data that is automatically shared with external partner accounts.

Detailed Analysis:

- PII Data Flow:**
 - The lambda function (`infrastructure/data-processing/lambda/lambda_function.py:95`) now includes user email addresses in processed data
 - This data is stored in the analytics S3 bucket (`data.terraform_remote_state.analytics.outputs.analytics_bucket_id`)
 - The analytics bucket has cross-account access configured for partner account `927374824227`
- Partner Account Access:**
 - The analytics S3 bucket policy (`infrastructure/analytics/modules/s3-analytics-bucket/main.tf:114-133`) grants the partner account:
 - `s3:GetObject` - Can read all files including PII data
 - `s3:ListBucket` - Can list all objects in the bucket
 - `s3:GetBucketLocation` and `s3:GetBucketVersioning` - Metadata access
 - Partner account has ROOT access (`arn:aws:iam::927374824227:root`) with minimal restrictions
- Data Storage Location:**
 - PII data is stored in path: `user-data/{environment}/year={year}/month={month}/day={day}/user_data_{timestamp}.json`
 - Partner account can access ALL historical and future PII data

Lock conversation

Diapositiva traducida / resumen visual:

Los asistentes modernos combinan razonamiento + contexto + herramientas.

Ideas clave del vídeo

- Claude destaca especialmente por su uso de herramientas.
- Puede automatizar tareas complejas de desarrollo.
- Comprende contexto e infraestructura.
- Es extensible mediante integraciones personalizadas.
- Puede detectar riesgos de seguridad antes del despliegue.